



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Rene Adrián Davila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 5

Integrante(s): 323001315, 426057857, 323008754, 426058665, 323067267

*No. de lista o
brigada:*

5

Semestre: 2027-1

Fecha de entrega: 2 de Octubre

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
1.0.1. Objetivos específicos	2
2. Marco Teórico	2
2.1. Programación Orientada a Objetos	2
2.2. Abstracción	3
2.3. Encapsulamiento	3
2.4. Modificadores de acceso	3
2.5. Métodos consultores y modificadores	3
2.6. Ocultamiento de la implementación	4
2.7. Relación entre abstracción y encapsulamiento	4
3. Desarrollo	4
4. Resultados	7
4.1. Programa Empleado	7
4.2. Programa Fecha	8
4.3. Investigación con IA Gen	8
5. Conclusiones	10

1. Introducción

Este apartado debe abordar los siguientes puntos:

- **Planteamiento del problema.** En el desarrollo de programas orientados a objetos es necesario representar problemas del mundo real mediante clases que contengan únicamente las características y funcionalidades necesarias para resolverlos. Sin una adecuada organización, las clases pueden contener información innecesaria o permitir un acceso directo a sus datos, lo que puede afectar la estructura y seguridad del programa. La Práctica 5, denominada *Abstracción y encapsulamiento*, plantea el diseño de clases a partir de las características y funcionalidades principales de un objeto, así como el uso de diferentes niveles de acceso para proteger la información y ocultar los detalles de implementación.
- **Motivación.** La aplicación de la abstracción permite concentrarse en los aspectos relevantes de un problema sin considerar todos sus detalles internos. Por otra parte, el encapsulamiento permite proteger los datos de los objetos y establecer mecanismos controlados para acceder o modificar su información. Estos conceptos son importantes para desarrollar clases organizadas y facilitar el mantenimiento de los programas orientados a objetos.
- **Objetivos.** Aplicar los conceptos de abstracción y encapsulamiento en el diseño de clases mediante Java, identificando las características y funcionalidades principales de los objetos y utilizando niveles de acceso adecuados para proteger su información.

1.0.1. Objetivos específicos

- Identificar las características principales de los objetos que forman parte de un problema.
- Determinar las funcionalidades necesarias para representar dichos objetos.
- Aplicar el concepto de abstracción en el diseño de clases.
- Utilizar diferentes niveles de acceso para controlar la visibilidad de atributos y métodos.
- Aplicar el encapsulamiento para proteger la información de los objetos.

2. Marco Teórico

2.1. Programación Orientada a Objetos

La Programación Orientada a Objetos es un paradigma que permite representar un problema mediante objetos que poseen características y comportamientos. Para ello, las clases establecen la estructura que tendrán los objetos y permiten organizar sus datos y operaciones de acuerdo con las necesidades del problema [2].

Dentro de este paradigma existen diferentes conceptos que permiten organizar y proteger la información de los objetos. Entre ellos se encuentran la abstracción y el encapsulamiento, los cuales permiten simplificar la representación de un problema y controlar el acceso a sus datos.

2.2. Abstracción

La abstracción consiste en identificar los aspectos más importantes de un problema y representarlos sin considerar aquellos detalles que no son necesarios para su solución. De esta manera, permite trabajar con un nivel de representación más sencillo y concentrarse en lo que hace un objeto en lugar de enfocarse inicialmente en cómo realiza sus operaciones [1].

En el diseño de clases, la abstracción permite determinar cuáles son las características y funcionalidades que realmente necesita tener un objeto. Esto ayuda a evitar que una clase contenga información o comportamientos que no sean relevantes para el problema que se está resolviendo.

2.3. Encapsulamiento

El encapsulamiento consiste en proteger los datos internos de un objeto y controlar la manera en que el resto del programa puede acceder a ellos. En lugar de permitir que otras clases modifiquen directamente los atributos, el acceso puede realizarse mediante los métodos definidos por el propio objeto [1].

Este mecanismo permite mantener un mayor control sobre la información de una clase y ocultar detalles de su implementación. Por lo tanto, los datos pueden mantenerse protegidos mientras se proporcionan las operaciones necesarias para utilizarlos.

2.4. Modificadores de acceso

Los modificadores de acceso permiten establecer la visibilidad de los atributos y métodos de una clase. En Java se utilizan principalmente los modificadores `public`, `protected` y `private`. También existe un nivel de acceso por defecto cuando no se especifica alguno de estos modificadores [1].

El modificador `private` restringe el acceso directo a los miembros de una clase, mientras que `public` permite que puedan ser utilizados desde otras clases. El modificador `protected` proporciona acceso dentro de la propia clase y permite su utilización en clases derivadas [1].

2.5. Métodos consultores y modificadores

Cuando los atributos de una clase se mantienen protegidos, es necesario proporcionar mecanismos para consultar o modificar sus valores. Para ello pueden utilizarse métodos consultores y métodos modificadores.

Los métodos consultores permiten obtener información de los atributos sin modificar sus valores. En Java suelen utilizarse métodos con el prefijo `get`, conocidos como

getters. Por otro lado, los métodos modificadores permiten establecer o cambiar los valores de los atributos y suelen utilizar el prefijo **set**, conocidos como *setters* [1]. El uso de estos métodos permite mantener los atributos protegidos y controlar las operaciones que pueden realizarse sobre ellos. De esta forma, es posible validar o restringir los valores que recibe un atributo antes de modificarlo.

2.6. Ocultamiento de la implementación

El ocultamiento de la implementación consiste en evitar que otras partes del programa necesiten conocer los detalles internos de una clase para utilizarla. La clase proporciona una interfaz mediante sus métodos, mientras que la forma en que realiza internamente sus operaciones permanece protegida [1].

Esto permite modificar la implementación interna de una clase sin afectar necesariamente a las demás partes del programa que utilizan sus servicios. Por ello, el ocultamiento de la implementación contribuye a una mejor organización y mantenimiento del software.

2.7. Relación entre abstracción y encapsulamiento

La abstracción y el encapsulamiento son conceptos relacionados, pero cumplen funciones diferentes. La abstracción permite identificar y representar los aspectos relevantes de un problema, mientras que el encapsulamiento permite proteger los datos y controlar el acceso a la implementación de los objetos.

La aplicación conjunta de ambos conceptos permite diseñar clases que representen únicamente la información y funcionalidades necesarias, manteniendo protegidos sus datos internos y proporcionando métodos para interactuar con ellos de manera controlada. ““

3. Desarrollo

Ejercicio 1: Consulta sobre Métodos Modificadores en Constructores

Desarrollo de la Investigación en Equipo

- **Inicio de la investigación en equipo:** Cada uno de los 5 integrantes realizó una consulta independiente con su respectiva IA sobre la conveniencia de utilizar métodos modificadores (*setters*) dentro de los constructores de una clase.
- **Consulta del Integrante 1:** Se investigó sobre el principio de reutilización de código y la centralización de reglas de validación en la inicialización.
- **Consulta del Integrante 2:** Se indagó sobre los riesgos de herencia, polimorfismo y llamadas a métodos sobrescribibles dentro de un constructor.

- **Consulta del Integrante 3:** Se exploraron las pautas de inmutabilidad y la seguridad frente a entornos multihilo (*thread-safety*).
- **Consulta del Integrante 4:** Se analizó el Principio de Responsabilidad Única y la separación entre la construcción y la modificación de un objeto.
- **Consulta del Integrante 5:** Se investigaron alternativas de diseño limpio, como métodos validadores privados/estáticos o patrones de creación.
- **Comparación de resultados entre integrantes:** Se reúnen los aportes obtenidos por cada miembro del equipo para contrastar las diferentes perspectivas expuestas por las distintas IAs.
- **Identificación de hallazgos relevantes:** Se analizan las coincidencias entre los resultados para llegar a una postura unificada sobre la validez y correcta aplicación de los setters dentro de un constructor.

Ejercicio 2: Clase Empleado

Paso a Paso del Algoritmo

1. **Definición de la Clase Empleado:**
 - Declarar los atributos privados: `nombre (String)`, `apellido (String)` y `salarioMensual (double)`.
2. **Implementación del Constructor:**
 - Recibir tres parámetros para inicializar el nombre, apellido y salario mensual.
 - Asignar el salario mensual previa comprobación de que sea un valor positivo (> 0.0).
3. **Encapsulamiento y Métodos Get/Set:**
 - Definir métodos de acceso y modificación para cada atributo.
 - En `setSalarioMensual(double salario)`, verificar que el valor sea estrictamente mayor a 0.0; de lo contrario, mantener el valor actual.
4. **Cálculo de Salario Anual:**
 - Implementar el método `getSalarioAnual()` retornando el producto de `salarioMensual` por 12.
5. **Clase Principal (MainEmpleado):**
 - Instanciar dos objetos de la clase `Empleado` mediante la lectura de datos con `Scanner`.

- Imprimir el salario anual inicial de cada empleado.
- Aplicar un incremento del 10 % a cada salario e imprimir el salario anual actualizado.

Diagrama de clases – Empleado

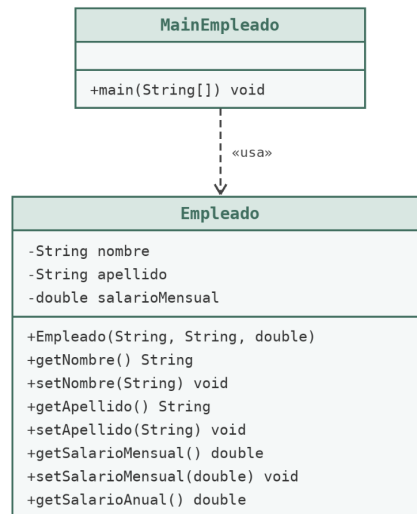


Figura 1: Diagrama de clases ejercicio 2

Ejercicio 3: Clase Fecha

1. Definición de la Clase Fecha:

- Declarar los atributos privados enteros: **mes**, **día** y **año**.

2. Implementación del Constructor:

- Recibir tres valores para inicializar el mes, día y año.
- Inicializar el mes, día y año asumiendo que los valores proporcionados por el usuario son correctos, tal como se solicitó.

3. Métodos de Acceso y Modificación:

- Implementar los métodos getter y setter para cada uno de los atributos.

4. Formateo y Visualización:

- Crear el método `mostrarFecha()` que imprima en consola los valores con el formato **mes/día/año**.

5. Clase Principal (MainFecha):

- Leer los valores correspondientes desde la entrada estándar.
- Instanciar la clase `Fecha` y llamar al método `mostrarFecha()` para comprobar su ejecución.

Diagrama de clases – Fecha

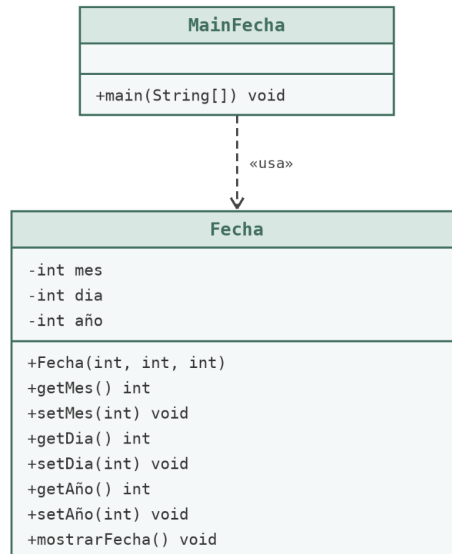


Figura 2: Diagrama de clases Ejercicio 3

4. Resultados

4.1. Programa Empleado

1-En la primera parte, correspondiente a la ejecución de la clase `MainEmpleado`, se verifica la creación de dos objetos inicializados dinámicamente con los datos ingresados por el usuario mediante la entrada estándar (Juan Perez y Ana Salgado). A partir de estos datos, se observa el cálculo correcto de sus respectivos salarios anuales iniciales. Posteriormente, se comprueba el correcto funcionamiento de los métodos modificadores al aplicarles de manera exitosa un aumento del 10 % a su salario mensual, lo que se refleja en los nuevos montos anuales impresos en la pantalla.

```

PS C:\Users\acer\Desktop\P00\PRACTICA5> java MainEmpleado
--- Empleado 1 ---
Nombre: Juan
Apellido: Perez
Salario mensual: 4500

--- Empleado 2 ---
Nombre: Ana
Apellido: Salgado
Salario mensual: 5400

--- Salario Anual Inicial ---
Juan Perez: $54000.0
Ana Salgado: $64800.0

--- Salario Anual con Aumento del 10% ---
Juan Perez: $59400.0
Ana Salgado: $71280.000000000001
PS C:\Users\acer\Desktop\P00\PRACTICA5>

```

Figura 3: Ejecución de las clases MainEmpleado.

4.2. Programa Fecha

2-En la segunda parte, correspondiente a la ejecución de la clase MainFecha, se verifica la creación de un objeto inicializado dinámicamente con los datos proporcionados por el usuario mediante la entrada estándar, la cual se imprime con el formato requerido separado por diagonales (10/3/2026). Finalmente, se demuestra el uso de los métodos setters y getters para alterar y leer individualmente el mes, día y año, culminando con la impresión de la nueva fecha actualizada (12/25/2027).

```

PS C:\Users\acer\Desktop\P00\PRACTICA5> java MainFecha
Mes: 10
Día: 3
Año: 2026
La fecha inicial es: 10/3/2026

--- Fecha modificada ---
Mes: 12
Día: 25
Año: 2027
Fecha completa actualizada: 12/25/2027
PS C:\Users\acer\Desktop\P00\PRACTICA5>

```

Figura 4: Ejecución de las clases MainFecha.

4.3. Investigación con IA Gen

A primera vista, invocar métodos modificadores (comúnmente conocidos como *setters*) dentro del constructor de una clase parece una decisión lógica y conveniente. El razonamiento inicial de muchos desarrolladores es evitar la duplicación de código: si el método *setter* ya contiene la lógica de validación de los datos (por ejemplo, verificar

que una edad no sea negativa o que un texto no esté vacío), ¿por qué no reutilizarlo al momento de instanciar el objeto? Sin embargo, en el paradigma de la Programación Orientada a Objetos (POO), el uso de métodos modificadores públicos o sobrescribibles dentro de los constructores es considerado una mala práctica (anti-patrón) que puede introducir errores sutiles, difíciles de rastrear y de depurar.

A continuación, se presenta una justificación detallada de por qué esta práctica resulta inconveniente, abarcando los riesgos de diseño, seguridad y estabilidad del código:

1. El peligro de los métodos virtuales y la inicialización parcial

El problema más grave de llamar a un método *setter* en un constructor ocurre en lenguajes que soportan polimorfismo y herencia (como Java, C#, C++, entre otros). Cuando una clase base llama a un método modificador en su constructor, está invocando un método que potencialmente puede ser sobrescrito (*overridden*) por una clase hija o subclase.

El ciclo de vida de creación de un objeto dicta que el constructor de la clase padre se ejecuta antes que el constructor de la clase hija. Si el constructor del padre llama a un *setter*, y la clase hija ha sobrescrito ese *setter*, se ejecutará la versión del método de la clase hija. El problema crítico aquí es que el *setter* de la clase hija se ejecutará en un momento en el que la clase hija aún no ha sido inicializada (su constructor no ha terminado, ni siquiera ha empezado). Si el *setter* de la hija intenta acceder a variables específicas de la subclase, encontrará valores nulos o basura (dependiendo del lenguaje), lo que provocará excepciones (como un `NullPointerException` en Java) o comportamientos erráticos.

2. Imposibilidad de garantizar la inmutabilidad

Una de las mejores prácticas en la ingeniería de software moderna es favorecer la inmutabilidad de los objetos. Un objeto inmutable es aquel cuyo estado no puede cambiar una vez que ha sido creado (por ejemplo, usando `final` en Java, `readonly` en C# o `const` en C++). Los objetos inmutables son inherentemente seguros en entornos multihilo (*thread-safe*), más fáciles de probar y menos propensos a errores secundarios.

Si decides usar métodos *setters* para inicializar las variables en el constructor, te ves obligado a declarar dichas variables como mutables. En lenguajes como Java, una variable `final` solo puede ser asignada directamente en el cuerpo del constructor (o en su declaración). Un método *setter*, por su propia naturaleza, no puede asignar valor a una variable `final`. Por lo tanto, usar *setters* te priva de la capacidad de crear objetos inmutables y seguros, forzándote a un diseño más frágil.

3. Confusión de responsabilidades (Principio de Responsabilidad Única)

Desde una perspectiva semántica y de diseño, un constructor y un método modificador tienen propósitos completamente distintos.

- El constructor tiene la responsabilidad exclusiva de transicionar un objeto desde la “nada” hacia un estado inicial válido.

- El método modificador (*setter*) tiene la responsabilidad de transicionar un objeto que ya está en un estado válido hacia otro estado válido.

Mezclar estas responsabilidades ensucia el contrato de la clase. Además, un *setter* a menudo dispara eventos secundarios (como notificar a la interfaz de usuario que un valor cambió, registrar *logs* o actualizar bases de datos). Disparar estos eventos secundarios cuando el objeto ni siquiera ha terminado de construirse (durante la ejecución del constructor) suele causar efectos colaterales indeseados en la aplicación.

4. Alternativas recomendadas: ¿Cómo evitar la duplicación de código?

Si el objetivo de usar un *setter* en el constructor era reutilizar la lógica de validación, existen enfoques mucho más seguros y elegantes para lograrlo sin comprometer la integridad del objeto:

- **Métodos de validación privados/estáticos:** Extrae la lógica de validación (el *if* que comprueba si la edad es válida, por ejemplo) a un método *private* (y preferiblemente *static* o *final*). Llama a este método validador tanto desde el constructor como desde el *setter*. De esta forma, el constructor hace la asignación directa de la variable, pero reutiliza la lógica de comprobación.
- **Uso del Patrón de Diseño Builder o Factory Method:** Si la creación del objeto requiere validaciones muy complejas o la instanciación de múltiples parámetros, es preferible delegar esta lógica a una clase constructora (*Builder*) o a un método de fábrica. Estos patrones se encargan de validar todos los datos antes de invocar al constructor, permitiendo que el constructor de la clase sea limpio, simple y asigne valores directamente a variables inmutables.

En general:

Aunque a corto plazo puede parecer un “atajo” conveniente para no escribir dos veces una validación, invocar métodos modificadores dentro de los constructores rompe los principios de encapsulamiento seguro, abre la puerta a errores catastróficos durante la herencia y bloquea la posibilidad de usar atributos inmutables. La buena arquitectura de software exige que los constructores hagan su trabajo de forma autónoma y directa, dejando la mutabilidad para una etapa posterior en la vida útil del objeto.

5. Conclusiones

La realización de la Práctica 5 permitió comprender la importancia de la abstracción y el encapsulamiento en el desarrollo de programas orientados a objetos. La abstracción facilitó identificar las características y funcionalidades necesarias de los objetos, evitando incluir información que no fuera relevante para la solución.

Por otra parte, el encapsulamiento permitió proteger los datos de las clases mediante el uso de diferentes niveles de acceso y métodos para controlar su consulta y modificación. En conjunto, estos conceptos permiten desarrollar clases mejor organizadas, con una estructura más clara y un acceso controlado a la información, facilitando así el mantenimiento y la comprensión del programa.

Referencias

- [1] M. A. Nakayama C. y J. A. Solano Gálvez, *Manual de prácticas del Laboratorio de Programación Orientada a Objetos*, Guía práctica de estudio 05: Abstracción y encapsulamiento, Facultad de Ingeniería, UNAM, 2017.
- [2] P. Deitel y H. Deitel, *Cómo programar en Java*, 7.a ed. México: Pearson Educación, 2008.
- [3] D. Barnes y M. Kölling, *Programación Orientada a Objetos con Java*, 3.a ed. Madrid: Pearson Educación, 2007.
- [4] L. Joyanes, *Fundamentos de programación. Algoritmos, estructuras de datos y objetos*, 4.a ed. México: McGraw-Hill, 2008.